CPSC 4970 – C++ Programming
Homework 3

Your solution to each problem should include the C++ source code and the screenshot of a test run. A solution without a test run will have points deducted.

1. (**Driver's license exam**) The local Driver's License Office has asked you to write a program that grades the written portion of the driver's license exam. The exam has 20 multiple-choice questions. Here are the correct answers:

1. A	6. B	11. A	16. C
2. D	7. A	12. C	17. C
3. B	8. B	13. D	18. A
4. B	9. C	15. B	19. D
5. C	10. D	15. D	20. B

Your program should store the correct answers shown above in an array. It should ask the user to enter the student's answers for each of the 20 questions, and the answers should be stored in another array. After the student's answers have been entered, the program should display a message indicating whether the student passed or failed the exam. (A student must correctly answer 15 of the 20 questions to pass the exam.) It should then display the total number of correctly answered questions, the total number of incorrectly answered questions, and a list showing the question numbers of the incorrectly answered questions.
Input Validation: Only accept the letters A, B, C, or D as answers.

2. (**Finding the index of the smallest element**) Write a function that returns the **index** of the smallest element in an array of integers. If the number of such elements is greater than 1, return the smallest index. Use the following header:

```
int indexOfSmallestElement(int a[], int size)
```

Write a test program that prompts the use to enter 10 numbers, invokes this function to return and display the index of the smallest element.

3. (**Counting occurrence of numbers**) Write a program that reads 10 integers between 0 and 99 and counts the occurrences of each. Here is a sample run of the program:

```
Enter 10 integers between 0 and 99: 2 5 6 5 4 3 23 43 2 56
2 occurs 2 times
3 occurs 1 time
4 occurs 1 time
5 occurs 2 times
6 occurs 1 time
23 occurs 1 time
43 occurs 1 time
56 occurs 1 time
```

(Hint: Use an array of 100 integers, say ***counts***, to store the counts for the number of 0s, 1s, ..., 99s.)

4. (2D Arrays) Write a program that creates a two-dimensional int array initialized with test data. You may choose any dimensions you wish. The program should have the following functions:

- **getHighestInRow** – This function should accept a two-dimensional array and an integer as its arguments. The integer specifies a row in the array. The function should return the highest value in that row.
- **getLowestInColumn** – This function should accept a two-dimensional array and an integer as its arguments. The integer specifies a column in the array. The function should return the lowest value in that column.

Demonstrate each of the functions in this program.

5. (Tic-Tac-Toe) Write a program that allows two players to play a game of tic-tac-toe. Use a two-dimensional char array with three rows and three columns as the game board. Each element of the array should be initialized with an asterisk (*). The program should run a **loop** that does the following:

- Display the contents of the board array.
- Allows player 1 to select a location on the board for an X. The program should ask the user to enter the row and column numbers.
- Allows player 2 to select a location on the board for an O. The program should ask the user to enter the row and column numbers.
- Determines whether a player has won, or a tie has occurred. If a player has won, the program should declare that player the winner and end. If a tie has occurred, the program should display an appropriate message and end.

Player 1 wins when there are three Xs in a row on the game board. The Xs can appear in a row, in a column, or diagonally across the board. Player 2 wins when there are three Os in a row on the game board. The Os can appear in a row, in a column, or diagonally across the board. A tie occurs when all of the locations on the board are full, but there is no winner.

6. (Test Scores – Vectors) Write a program that uses a STL vector to hold a user-defined number of test scores. Once all the scores are entered, your program should calculate and display the average score.

7. (Password – Character Testing) Imagine you are developing a software package that requires users to enter their own passwords. Your software requires that users' passwords meet the following criteria:

- The password should be at least six characters long.
- The password should contain at least one uppercase and at least one lowercase letter.
- The password should have at least one digit.

Write a **function** that accepts a string as an argument and returns true if the string meets the password criteria, and false otherwise. Demonstrate the function in a program.

8. (Occurrences – Characters and Strings) Write a program that will read in a line of text and output the number of occurrences of each letter. Upper- and lowercase versions of a letter are counted as the same letter. Output the letters in alphabetical order and list only those letters that occur in the input line. For example, the input line

```
I say Hi.
```

should produce the following output:

```
1  a

1  h

2  i

1  s

1  y
```

9. (Word Separator) Write a program that accepts as input a sentence in which all of the words are run together, but the first character of each word is uppercase. Convert the sentence to a string in which the words are separated by spaces and only the first word starts with an uppercase letter. For example, the string “StopAndSmellTheRoses.” would be converted to “Stop and smell the roses.”